

7. Vue项目实战（一）项目评审与架构设计

课程资料

课前备课源码

byelide.zip
1.64MB



课中手把手实现源码

byelide-1.zip
357.44KB



课程源码，重点看这几个分支：

- features/base-config
- features/base-app-layer

使用工具：sourcetree

没怎么用过 git 的同学，用 sourcetree、vscode 自带的

课程目标



- 初级：
 - 了解无代码平台核心业务模型
 - 了解可视化相关架构设计思想
 - 掌握 Vue3 项目基础架构
- 中级：
 - 掌握 Vue3 项目工程化思维，包括打包构建流程
 - 掌握 Vue3 开发核心 API 及组件与模块化设计思想

- 了解项目规范与团队管理办法
- 高级：
 - 掌握前端企业及项目架构设计思想
 - 掌握从零到一设计架构一个产品
 - 掌握技术选型与评估，并能独立设计开发满足业务的组件以及封装库

注意：因为是实战课，除了看懂项目架构与基础开发以外，还要着重培养项目相关描述，在面试过程中这点非常重要！

本节课我们主要从业务层面出发，让大家深度参与一个项目从需求阶段到落地的整个实施过程。

 特别是之前很多同学没有参与过大型项目的，主要写后台相关 CRUD 的同学，一定一定要完全掌握本次实战课程的内容！这会直接决定你的编程思维与薪资涨幅！

课程大纲

1. 项目需求分析评审（入料、编排、渲染、出码）
2. Vue 项目基础架构设计，基于 Vite、Vue3、Pinia、Vue-Router
3. 构建基础框架（导航、物料、配置）
4. 编排的选型与实现（流式、画布式、Grid）

课程资料

课程目标

课程大纲

课程内容

项目需求分析评审（入料、编排、渲染、出码）

背景介绍

核心能力

架构设计

物料堆

编排

配置面板

渲染器与出码

扩展功能与进阶（这部分内容）

Vue 项目基础架构设计，基于 Vite、Pinia、Vue-Router

技术选型

项目初始化

规范约定

规范约束

课程内容

项目需求分析评审（入料、编排、渲染、出码）

背景介绍

近年来，无代码平台与可视化相关平台日益火热。

无代码允许开发人员低成本通过拖拉拽的方式快速构建企业内部系统或落地页，常见的无代码平台能力包括：审批流、应用构建等。其在企业开发提效与运营方面取得了不错的成绩。例如国外的：Coda、Airtable、Notion 等产品，例如国内的：明道云、极简云、ClickPaaS 等产品。构建无代码平台是一件有挑战性的事情，这节实战课会从业务与技术架构入手，带你彻底弄懂无代码平台研发。

可视化平台允许数据分析人员通过拖拉拽的方式快速组建仪表盘或数据分析大盘，其在企业运营方面发挥重要作用。例如国外的：PowerBI、Monday、Coda 等产品，国内的：观远数据、BDP、FineBI 等产品。构建可视化 SaaS 是一件非常有趣的事情，这节实战课会从业务与技术架构出发，带你从零完成可视化图表数据协议设计、图表渲染器开发、图表编排等功能。

当无代码遇上可视化，相信会有更多火花，没错，我们课程将这两大火热业务方向进行融合，带你真正体验大厂企业级项目的设计、开发与流程管控。

开发过程中，我们会严格按照大厂开发流程，从项目搭建、规范约定、技术选型与评估、需求分析到开发，每个流程细节我们都不会放过。技术选型方面，我们会选用 Vue3 作为技术栈，状态管理选用 Pinia，编辑器开发我们会基于 tiptap 来展开，Echarts 开发图表相关内容，除此之外，我们会向大家展示如何更优雅更松散处理复杂表单的场景。相信你学完后，一定会感慨自己就这样又更上了一层楼！

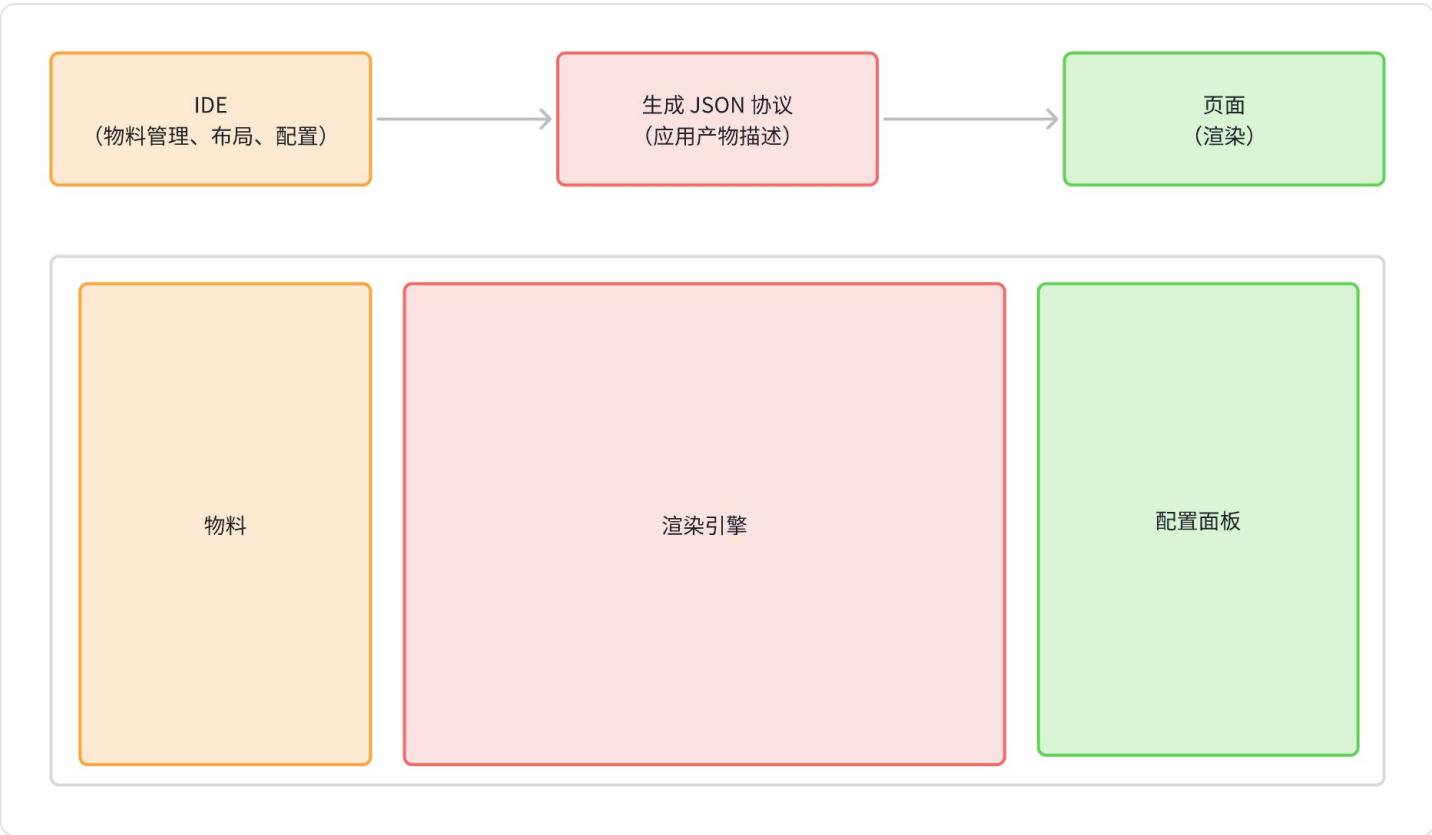
💡 请注意无代码与低代码的主要区别：

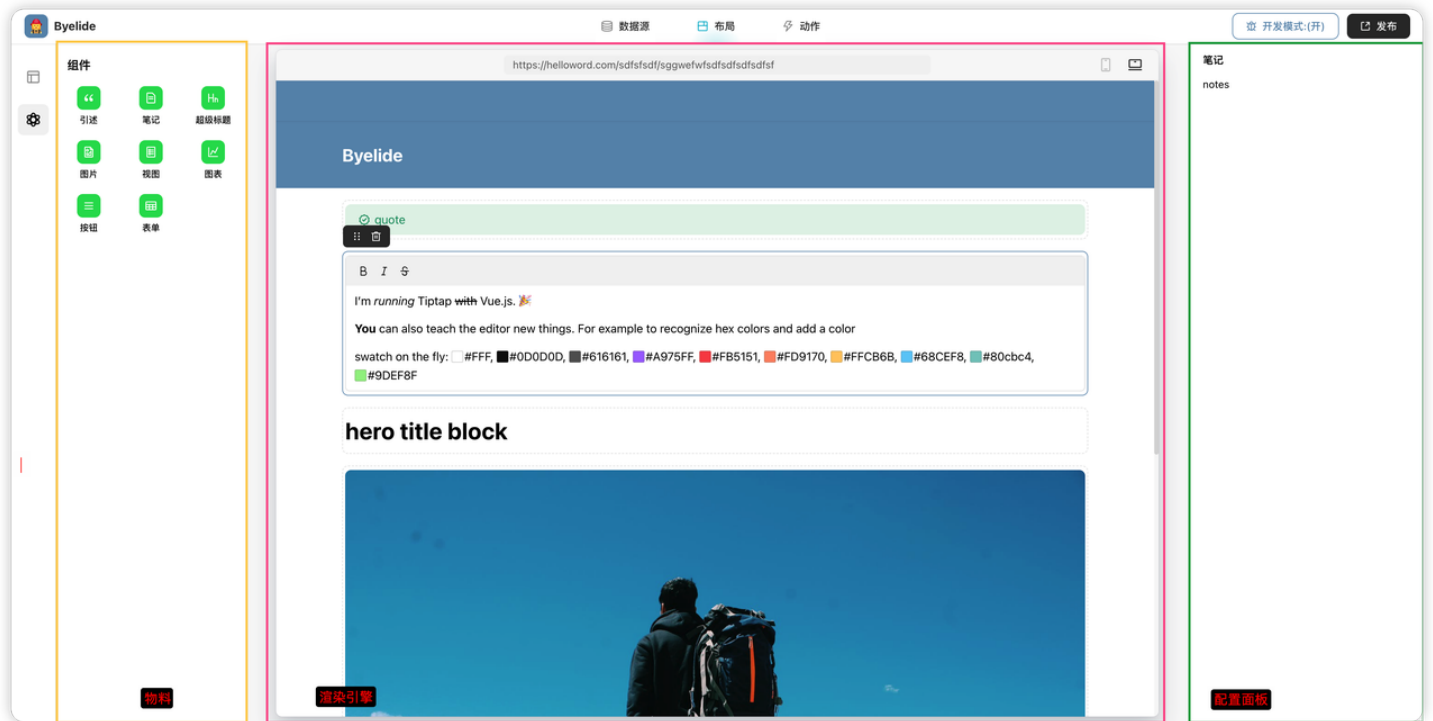
- 1. 无代码一般面向普通用户，可以没有编程经验，可以以最简单形式搭建应用。而低代码需要用户具备一定的编程能力。
- 2. 在实现上，低代码比无代码更复杂，低代码还会存在例如 DSL 以及代码植入的能力，低代码的代码片段植入思路接近微前端。

核心能力

- 可视化配置面板
- 具有可拓展能力：组件、模板、逻辑复用
- 生命周期管理：开发管理、页面管理、部署管理

架构设计





物料堆

包含可供使用的物料，通常物料的组织与消费是插件化的，还有很多产品将物料组织发布到插件市场，可供用户自行拓展，这都是跟插件化思想与 **js 沙盒** 设计离不开的。

<https://juejin.cn/post/6981374562877308936>

编排

- 微内核思想：操作的是DSL（json树）， $f(\text{state}) \rightarrow \text{view}$
 - 组合和渲染层隔离
 - $\text{render runtime}(\text{渲染引擎sdk}) + \text{dsl}(\text{json}) = \text{页面}$
- 事件：DND拖拽、Mouse Event
- 画布分层技术：借鉴浏览器渲染模型，使用冒泡机制，走到所有层
 - 第一层：div，负责渲染， $\text{render}(\text{dsl}, \text{document.querySelector}(\text{"\#root"}))$
 - 第二层：加一层div，只负责处理右键事件
 - 第三层：加一层div，只处理快捷键
 - event bus：所有层可通过 event bus 进行通信
- 画布功能及拓展：
 - 简单编排
 - 物料均为块级，操作简便，不涉及复杂布局问题
 - 画布灵活编排

- 无限画布、引导线（衫格）、吸附对齐、旋转、快捷键、右键、缩放
- 无限画布：监听滚动事件，每次给画布加宽带
- 引导线：使用div画线（高度和宽带为1px）、绝对定位可拖动，下方
- 吸附对齐：计算想尽的dom节点，定6个点和3个线，距离相近时，设置距离为0

配置面板

配置面板就是对物料进行配置，我们需要将数据、视图、操作解耦

- 功能：撤销、重做、预览、提测、发布
 - 重做（undo/redo）：使用队列，指针移动
 - 预览：render(dsl: {type, key, props {}, animate {}, actions: {}, attrs: {}, children []})
- 中间层：权限控制、数据操作（转换），暴露部分api

这块比较考验大家对于复杂表单的设计实现

渲染器与出码

通常平台会提供 schema 渲染页面的功能，同时也可将所有配置输出 JSON Schema，我们称为**出码**功能

~~扩展功能与进阶（这部分内容）~~

~~历史记录和版本、模版、分享、主题~~

~~进阶：协同编辑、定时任务、微前端（集成到其他系统的能力）、混合开发~~

- ~~混合开发：组件（ts、flow）和json混合开发~~
- ~~逻辑的配置：使用流程图（flow），最后生成业务逻辑~~
- ~~协同：crdt算法，yjs、ot等方案~~

Vue 项目基础架构设计，基于 Vite、Pinia、Vue-Router

技术选型

- 包管理：pnpm
- 工程化相关
 - vite

- lint-staged
- cspell
- commitizen
- cz-git
- husky
- zx
- tsno
- lint规范：
 - commitlint;
 - stylelint;
 - prettier;
 - eslint;
 - editorconfig;
- Vue CLI、Vue3
- Pinia
- Vue-Router
- 拖拽库：基于原生的 smooth-dnd 封装用于 Vue3 的拖拽组件
- 编辑器：**tiptap** for vue3, <https://tiptap.dev/installation/vue3>
 - ice
 - proseMirror, tiptap 就是基于 proseMirror 的上层封装
 - <https://github.com/facebook/lexical>
- UI库：Arco Design, <https://arco.design/vue/docs/start>
- 表单校验：vee-validate, <https://vee-validate.logaretm.com/v4/>
- 流程编排：@vue-flow/core, <https://vueflow.dev/>
- 图表：echarts、zerender、d3.js

项目初始化

我们使用 `create-vue` 初始化基础结构，在此基础上设计整个项目架构。

```
1 npm init vue@latest
2
```

规范约定

如果你作为前端 Leader 第一需要考虑的就是开发实现成本，但是往往每个成员开发的方式各异，因此我们在项目早期就一定需要做足规范方面的约束。

通常规范约束包含：

- 代码相关：js/ts/style 编码规范
- 命名与单词拼写规范
- git 提交规范等

基于此，你需要设计完整的规范约定，并在编码和代码提交时做检测，检测不通过不允许代码提交。

规范约束

代码相关检查我们使用 eslint 约束 ts 编码，stylelint 约束样式相关。

eslint 项目脚手架已经集成我们不做过多介绍，stylelint 的使用比较简单：

安装 stylelint

修改 `package.json`

```
1  "script": {
2    "lint:style": "stylelint **/*.{vue,css}",
3  },
4  "devDependencies": {
5    "stylelint": "15.10.2",
6    "stylelint-config-standard": "34.0.0",
7    "stylelint-config-prettier": "9.0.5",
8    "stylelint-config-html": "1.1.0",
9    "stylelint-config-vue": "1.0.0",
10  },
```

注意，需要增加关于 stylelint 的配置文件

`stylelint.config.js`

```
1  /* eslint-env node */
2  module.exports = {
3    extends: ['stylelint-config-standard', 'stylelint-config-prettier',
4             'stylelint-config-html/vue']
```

```
4   }
5
```

安装 cspell

```
1  "script": {
2    "spellcheck": "cspell lint --dot --gitignore --color --cache --show-
  suggestions \"src/**/*.@(html|js|cjs|mjs|ts|tsx|css|scss|md|vue)\",
3  },
4  "devDependencies": {
5    "cspell": "6.31.2",
6  }
```

配置 cspell，需要在项目根目录创建 `cspell.json`

```
1  {
2    "import": ["@cspell/dict-lorem-ipsum/cspell-ext.json"],
3    "caseSensitive": false,
4    "dictionaries": ["custom-words"],
5    "dictionaryDefinitions": [
6      {
7        "name": "custom-words",
8        "path": "./.cspell/custom-words.txt",
9        "addWords": true
10     }
11   ],
12   "ignorePaths": ["**/node_modules/**", "**/dist/**", "**/lib/**",
13     "**/docs/**", "**/stats.html"]
14 }
```

并在项目根目录创建 `.cspell/custom-words.txt`，把那些你主观认为是对的单词放进去，比如：

```
1  behaviour
2  Byelide
3  commitlint
4  conventionalcommits
5  optimizelegibility
6  pinia
7  tiptap
8  vuedraggable
```

最后执行 `pnpm i`

这时，你就可以在本地运行对应 script 进行检测了，例如：

`pnpm spellcheck` 、 `pnpm lint:style`



"lint": "run-p lint:js type-check lint:style spellcheck"

commit 检测

husky 相关

为了在提交时进行代码检测，我们就需要使用 git 提交钩子进行处理，方便起见我们一般使用 `husky`，同时我们需要使用 `commitlint` 以及 `lint-stage` 相关来配置如何检测。

在 `package.json` 中配置相关依赖

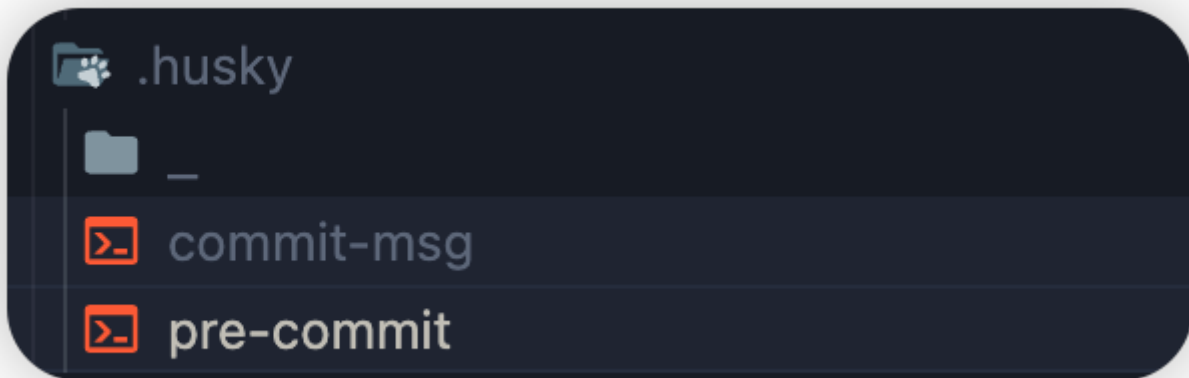
```

1  {
2    "script": {
3      "prepare": "husky install",
4      "lint:stage": "lint-staged",
5      "commit": "cz-git",
6      "commitlint": "commitlint --edit"
7    },
8    "lint-staged": {
9      "*.html,vue,css,sass,scss,ts": "prettier --write",
10     "*.vue,ts": "eslint --fix",
11     "*.vue,css,sass,scss": "stylelint --fix"
12   },
13   "config": {
14     "commitizen": {
15       "path": "node_modules/cz-git"
16     }
17   },
18   "devDependencies": {
19     "@commitlint/cli": "17.6.7",
20     "@commitlint/config-conventional": "17.6.7",
21     "@commitlint/cz-commitlint": "17.6.7",
22     "commitizen": "4.3.0",
23     "husky": "8.0.3",

```

```
24     "lint-staged": "13.2.3",
25     "cz-git": "1.7.0",
26     "zx": "2.0.0"
27   }
28 }
```

执行 `pnpm prepare` 就会在根目录生成 `.husky`，钩子相关的内容写在这里。



执行以下命令，实现 husky 提交前的约束 hook

```
1  npx husky add .husky/pre-commit
```

为了方便大家学习，我选用了最简单让大家写脚本的方案，我们都知道一般我们会选择使用 shell 脚本，但是很多同学没有学习过，我们可以选择使用 `zx` 它能让我们写 js 代码以执行脚本相关内容，比如上图看到的 `pre-commit`，我们是这样：

```
1  #!/usr/bin/env sh
2  . "$(dirname "$0")/_/husky.sh"
3
4  pnpm exec tsno run ./scripts/pre-commit.ts
```

在项目根目录下创建 `scripts/xxx` 具体脚本的内容我们在 `scripts/pre-commit.ts` 中编写。

```
1  #!/usr/bin/env zx
2
3  import { $ } from 'zx'
4
```

```

5 console.log('开始执行代码质量评估...\n')
6
7 await import('./check').catch((out) => {
8     throw new Error('代码质量评估失败, 请检查代码')
9 })
10
11 console.log('printf "检测通过, 创建 commit 中...\n')
12
13 await `$git add .`
14

```

./check.ts

```

1  #!/usr/bin/env zx
2
3  import type { ProcessOutput } from 'zx'
4  import { $ } from 'zx'
5  import { printObject } from './utils'
6
7  await `$pnpm spellcheck`.catch((out: ProcessOutput) => {
8      console.log(out)
9
10     throw new Error(out.stdout)
11 })
12
13 // await Promise.all([$`pnpm type-check`, `$pnpm lint`]).catch((out:
14 //   ProcessOutput) => {
15 //     printObject(out)
16 //     throw new Error(out.stdout)
17 // })
18
19 // check type and stage
20 await Promise.all([$`pnpm type-check`, `$pnpm lint:stage`]).catch((out:
21 ProcessOutput) => {
22     printObject(out)
23     throw new Error(out.stdout)
24 })
25

```

./utils

```

1  import { ProcessOutput } from 'zx/core'
2

```

```

3 export function printObject(
4   object: Record<string, unknown> | ProcessOutput,
5   method: 'log' | 'warn' | 'error' = 'log'
6 ) {
7   for (const [key, value] of Object.entries(object)) {
8     // eslint-disable-next-line no-console
9     console[method](` ${key}: \n${value} \n`)
10  }
11 }
12

```

commitlint 相关

另外, commitlint 及 git-cz 相关的配置我们需要在根目录创建 `commitlint.config.js`

```

1  // module.exports = { extends: ['@commitlint/config-conventional'] }
2
3  // eslint-disable-next-line no-undef
4  module.exports = {
5    extends: ['@commitlint/config-conventional'], // extends can be nested
6    parserPreset: 'conventional-changelog-conventionalcommits',
7    prompt: {
8      settings: {},
9      messages: {
10        skip: ':skip',
11        max: 'upper %d chars',
12        min: '%d chars at least',
13        emptyWarning: 'can not be empty',
14        upperLimitWarning: 'over limit',
15        lowerLimitWarning: 'below limit'
16      },
17      types: [
18        { value: 'feat', name: 'feat:      ✨ A new feature', emoji: '✨ ' },
19        { value: 'fix', name: 'fix:      🐛 A bug fix', emoji: '🐛 ' },
20        { value: 'docs', name: 'docs:     📖 Documentation only changes',
emoji: '📖 ' },
21        {
22          value: 'style',
23          name: 'style:    🛠 Changes that do not affect the meaning of the
code',
24          emoji: '🛠 '
25        },
26        {
27          value: 'refactor',
28          name: 'refactor: 🔄 A code change that neither fixes a bug nor adds
a feature',

```

```

29     emoji: '📦 '
30 },
31 {
32     value: 'perf',
33     name: 'perf: 🚀 A code change that improves performance',
34     emoji: '🚀 '
35 },
36 {
37     value: 'test',
38     name: 'test: 🛠️ Adding missing tests or correcting existing
tests',
39     emoji: '🛠️ '
40 },
41 {
42     value: 'build',
43     name: 'build: 🔧 Changes that affect the build system or
external dependencies',
44     emoji: '🔧 '
45 },
46 {
47     value: 'ci',
48     name: 'ci: 🏠 Changes to our CI configuration files and
scripts',
49     emoji: '🏠 '
50 },
51 {
52     value: 'chore',
53     name: 'chore: 🛠️ Other changes that don't modify src or test
files",
54     emoji: '🛠️ '
55 },
56 { value: 'revert', name: 'revert: ⏮️ Reverts a previous commit',
emoji: ':rewind:' }
57 ],
58 useEmoji: true,
59 confirmColorize: true,
60 emojiAlign: 'center',
61 questions: {
62     scope: {
63         description: 'What is the scope of this change (e.g. component or
file name)'
64     },
65     subject: {
66         description: 'Write a short, imperative tense description of the
change'
67     },
68     body: {

```

```

69         description: 'Provide a longer description of the change'
70     },
71     isBreaking: {
72         description: 'Are there any breaking changes?'
73     },
74     breakingBody: {
75         description:
76             'A BREAKING CHANGE commit requires a body. Please enter a longer
description of the commit itself'
77     },
78     breaking: {
79         description: 'Describe the breaking changes'
80     },
81     isIssueAffected: {
82         description: 'Does this change affect any open issues?'
83     },
84     issuesBody: {
85         description:
86             'If issues are closed, the commit requires a body. Please enter a
longer description of the commit itself'
87     },
88     issues: {
89         description: 'Add issue references (e.g. "fix #123", "re #123".)'
90     }
91 }
92 }
93 }
94

```

pnpm commit

通过上面配置，以后的提交都通过这个命令就搞定了。

构建基础框架（导航、物料、配置）

Vue3 项目基础框架我们可以像如下形式组织，不过可以根据自己项目实际情况调整。

```

1  byelide
2  ├──.DS_Store
3  ├──.cspell
4  │   └──custom-words.txt
5  ├──.cspellcache
6  ├──.eslintrc.cjs
7  ├──.husky

```

```
8 | | | _
9 | | | | _husky.sh
10 | | | | _commit-msg
11 | | | | _pre-commit
12 | | | | _prettierignore
13 | | | | _prettierrc.json
14 | | | | _vite
15 | | | | _vscode
16 | | | | | _extensions.json
17 | | | | | _README.md
18 | | | | | _commitlint.config.js
19 | | | | | _cspell.json
20 | | | | | _cypress
21 | | | | | _cypress.config.ts
22 | | | | | _env.d.ts
23 | | | | | _index.html
24 | | | | | _package.json
25 | | | | | _pnpm-lock.yaml
26 | | | | | _public
27 | | | | | | _favicon.ico
28 | | | | | | _scripts
29 | | | | | | | _check.ts
30 | | | | | | | _pre-commit.ts
31 | | | | | | | | _utils.ts
32 | | | | | | _src
33 | | | | | | | | _DS_Store
34 | | | | | | | | | _App.vue
35 | | | | | | | | | _assets
36 | | | | | | | | | | | _base.css
37 | | | | | | | | | | | _logo.svg
38 | | | | | | | | | | | _main.css
39 | | | | | | | | | | | _reset.css
40 | | | | | | | | | | | | _variable.css
41 | | | | | | | | | _blocks
42 | | | | | | | | | _components
43 | | | | | | | | | _constants
44 | | | | | | | | | | | _blocksBaseMeta.ts
45 | | | | | | | | | _main.ts
46 | | | | | | | | | _mocks
47 | | | | | | | | | | | _blocks.ts
48 | | | | | | | | | _router
49 | | | | | | | | | | | _index.ts
50 | | | | | | | | | _setup.ts
51 | | | | | | | | | _stores
52 | | | | | | | | | | | _appEditor.ts
53 | | | | | | | | | | | _debug.ts
54 | | | | | | | | | _test.ts
```

```

55 |     |   └─types
56 |     |     |   └─block.ts
57 |     |     |     └─protocal
58 |     |     └─utils
59 |     |       └─array.ts
60 |     └─views
61 |   └─stylelint.config.js
62 |   └─tsconfig.app.json
63 |   └─tsconfig.json
64 |   └─tsconfig.node.json
65 |   └─tsconfig.vitest.json
66 |   └─vite.config.ts
67 |   └─vitest.config.ts

```

以上代码有 etree 生成，还不知道的同学可以试试

HOOKS

This command can run `commit-msg`, `prepare-commit-msg`, `pre-commit`, `post-commit` and `post-rewrite` hooks. See [githooks\[5\]](#) for more information.

主体框架

因为我们是单体应用，所以项目主题内容我们可以从路由 **router** 出发，路由决定了页面入口，路由分发到页面，通常与路由对应的页面我们存放在 **views** 中，组件相关内容在 **components** 内。

细心的同学可以发现，我们整个布局其实有很多细节，比如为什么页面外层容器需要 `position: fixed` 定位？

为什么边框我们选择使用 `box-shadow` 而不是直接使用 `border` ？

导航

```

1  <script setup lang="ts">
2  import { useEnvStore } from '@stores/debug'
3  import { Data, LayoutThree, Lightning, Share, Bug } from '@icon-park/vue-next'
4  import { computed, defineComponent, h } from 'vue'
5  import { useRoute } from 'vue-router'
6
7  const linkItems = [
8    {
9      value: 'dataSource',

```

```

10     label: '数据源',
11     bg: `radial-gradient(50% 50% at 50% 100%, rgba(0, 196, 83, 0.2) 0%,
    rgba(0, 196, 83, 0) 100%)`,
12     color: 'rgb(0, 196, 83)',
13     borderColor: 'radial-gradient(50% 50%, rgb(0, 196, 83) 0%, rgba(0, 196,
    83, 0) 100%)'
14   },
15   {
16     value: 'layout',
17     label: '布局',
18     bg: `radial-gradient(50% 50% at 50% 100%, rgba(24, 190, 212, 0.2) 0%,
    rgba(24, 190, 212, 0) 100%)`,
19     color: 'rgb(24, 190, 212)',
20     borderColor: 'radial-gradient(50% 50%, rgb(24, 190, 212) 0%, rgba(24,
    190, 212, 0) 100%)'
21   },
22   {
23     value: 'actions',
24     label: '动作',
25     bg: `radial-gradient(50% 50% at 50% 100%, rgba(241, 60, 11, 0.2) 0%,
    rgba(241, 60, 11, 0) 100%)`,
26     color: 'rgb(241, 60, 11)',
27     borderColor: 'radial-gradient(50% 50%, rgb(241, 60, 11) 0%, rgba(241, 60,
    11, 0) 100%)'
28   }
29 ]
30
31 defineProps<{
32   msg: string
33 }>()
34
35 // 千万不要这样!
36 // const { debug, toggle } = useEnvStore()
37 const envStore = useEnvStore()
38
39 const route = useRoute()
40 console.log('🚀 ~ file: AppNavigator.vue:35 ~ route:',
    JSON.parse(JSON.stringify(route)))
41
42 const activeLink = computed(() => route.name)
43
44 // 等价于 computed
45 // const activeLink = ref(route.path.slice(1))
46 // watch(
47 //   () => route.path,
48 //   (path) => {
49 //     console.log('🚀 ~ file: AppNavigator.vue:20 ~ path:', path)

```

```

50 //     activeLink.value = path.slice(1)
51 //   }
52 // )
53
54 const Icon = defineComponent({
55   setup(props) {
56     // 千万不能这样子写!!!
57     // const { type } = props
58     switch (props.type) {
59       case 'dataSource':
60         return () => h(Data, { size: 16 })
61       case 'layout':
62         return () => h(LayoutThree, { size: 16 })
63       case 'actions':
64         return () => h(Lightning, { size: 16 })
65
66       default:
67         return () => h('div')
68     }
69   },
70   props: {
71     type: {
72       type: String,
73       required: true
74     }
75   }
76 })
77 </script>
78
79 <template>
80   <div class="app-navigator">
81     <div class="app-info-wrapper">
82       <div class="app-logo">
83         
87       </div>
88       <h1 class="app-name">Byelide</h1>
89     </div>
90     <div class="app-navigator-link-wrapper">
91       <RouterLink
92         class="app-navigator-link-item"
93         v-for="item in linkItems"
94         :key="item.value"
95         :style="activeLink === item.value && { background: item.bg }"

```

```

95         :to="item.value"
96     >
97         <!-- defineComponent + h 代替条件渲染 -->
98         <!-- <div v-if="item.value === 'dataSource'"><Data /></div>
99         <div v-if="item.value === 'layout'"><LayoutThree /></div>
100        <div v-if="item.value === 'action'"><Lightning /></div> -->
101        <div
102            :style="{
103                lineHeight: 0.7,
104                color: activeLink === item.value ? item.color : 'var(--color-gray-
105                700)'"
106            >
107            <Icon :type="item.value" :active="activeLink === item.value" />
108        </div>
109        <span class="item-title">
110            {{ item.label }}
111        </span>
112        <div
113            class="item-border"
114            :style="activeLink === item.value ? { background: item.borderColor
115            } : {}"
116        ></div>
117        </RouterLink>
118    </div>
119    <div class="app-setting-wrapper">
120        <div class="common-btn debug-btn" :class="{ debug: envStore.debug }"
121        @click="envStore.toggle">
122            <Bug />
123            开发模式:({{ envStore.debug ? '开' : '关' }})
124        </div>
125        <div class="common-btn">
126            <Share />
127            发布
128        </div>
129    </div>
130 </template>
131 <style scoped>
132 .app-navigator {
133     position: relative;
134     z-index: 5;
135     display: flex;
136     justify-content: space-between;
137     align-items: center;
138     height: 48px;

```

```
139     box-shadow: rgb(0 0 0 / 8%) 0 1px 0;
140 }
141
142 .app-info-wrapper {
143     width: 1000px;
144     display: flex;
145     place-items: center;
146 }
147
148 .app-logo {
149     width: 32px;
150     height: 32px;
151     margin: 10px 8px 10px 18px;
152     border-radius: 8px;
153     background-color: var(--color-primary);
154 }
155
156 .app-logo img {
157     width: 100%;
158     height: 100%;
159     padding: 6px;
160 }
161
162 .app-name {
163     color: var(--color-gray-900);
164     font-weight: var(--font-weight-bolder);
165     font-size: var(--font-size-large);
166 }
167
168 .app-navigator-link-wrapper {
169     display: flex;
170     flex-shrink: 0;
171     justify-content: space-between;
172     align-self: stretch;
173 }
174
175 .app-navigator-link-item {
176     position: relative;
177     display: flex;
178     align-items: center;
179     padding: 0 8px;
180     margin: 0 24px;
181     align-self: stretch;
182     background-color: var(--color-white);
183     color: var(--color-text);
184     font-size: var(--font-size-normal);
185     font-weight: var(--font-weight-bold);
```

```
186 }
187
188 .item-title {
189     margin-left: 8px;
190     color: var(--color-black);
191 }
192
193 .item-border {
194     position: absolute;
195     bottom: -1px;
196     left: 0;
197     right: 0;
198     height: 1px;
199 }
200
201 .app-setting-wrapper {
202     width: 1000px;
203     display: flex;
204     place-items: center;
205     justify-content: flex-end;
206     margin-right: 18px;
207     gap: 12px;
208 }
209
210 .common-btn {
211     display: flex;
212     align-items: center;
213     gap: 8px;
214     padding: 6px 18px;
215     border-radius: 8px;
216     background-color: var(--color-black);
217     color: var(--color-white);
218     font-size: var(--font-size-normal);
219     font-weight: var(--font-weight-bold);
220     cursor: pointer;
221     user-select: none;
222 }
223
224 .debug-btn {
225     box-shadow: var(--color-gray-300) 0 0 0 1px;
226     background-color: var(--color-white);
227     color: var(--color-black);
228 }
229
230 .debug-btn.debug {
231     color: var(--color-primary);
232     box-shadow: var(--color-primary) 0 0 0 1px;
```

```
233   }
234   </style>
235
```

这块对于 `h` 运用以及如何将 Vue 组件策略渲染外部组件需要大家重点学习掌握。

物料

物料的处理主要是涉及到了布局以及拖拽，具体实现我们会在下次课为大家讲解拖拽库的封装实现。

编排的选型与实现（流式、画布式、Grid）

编排的实现方式有很多，我们可以选择基于画布式的自由拖拽、可以选择流式布局或者 grid，我们本次实战课选用的是流式。

这三种方式布局会考虑的技术给大家列举一下，可作为参考。

- 流式
 - smooth-dnd (<https://vueflow.dev/>)
 - vue.dragabble.next (<https://sortablejs.github.io/vue.draggable.next/#/simple>)
- 自由拖拽 (<https://kirillmurashov.com/vue-drag-resize/>)
- grid (<https://jbaysolutions.github.io/vue-grid-layout/>)

需求层面需要评判出来的优劣势

流式布局

- 没有脱离文档流，**自适应宽高**
- 缺点也很明显：不是平铺模式，后期可能存在比较严重的性能问题，另外对于内层数据的处理不方便。

```
1  {
2    type: 'text',
3    children: [
4      {
5        type: 'text',
6        children: [
7          type: 'text'
8        ]
9      }
10   ]
11 }
```

```
10     ]
11 }
```

自由拖拽，

- 脱离文档流了，布局更灵活
- 无法自适应宽高，如果实在需要自适应宽高，需要做非常复杂的处理

Grid

- 比较折中的办法，有一定的灵活性有一定的规范
- 无法自适应宽高

通过上面这些思考，我们最终确定了使用**流式布局**，接下来，问题又来了

- smooth-dnd (<https://vueflow.dev/>)
- vue.dragabble.next (<https://sortablejs.github.io/vue.draggable.next/#/simple>)

基于 smooth-dnd 的封装

因为 `smooth-dnd` 是原生js库，我们需要进行一定封装才能在 Vue3 项目中使用。

我们创建一个 SmoothDnd 组件，通常拖拽实现需要实现两个组件，分别为：Container 和 DraggableItem，我们分别定义两个组件，命名为 `SmoothDndContainer.ts` 和 `SmoothDndDraggable.ts`。

`SmoothDndContainer.ts`

```
1  import { defineComponent, h } from 'vue'
2  import { smoothDnD, dropHandlers } from 'smooth-dnd'
3  import type { SmoothDnD } from 'smooth-dnd'
4  import { getTagProps, validateTagProp } from './utils'
5
6  smoothDnD.dropHandler = dropHandlers.reactDropHandler().handler
7  smoothDnD.wrapChild = false
8
9  type EventKey = 'drag-start' | 'drag-end' | 'drop' | 'drag-enter' | 'drag-leave' | 'drop-ready'
10
11 const eventEmitterMap: Record<EventKey, string> = {
12   'drag-start': 'onDragStart',
13   'drag-end': 'onDragEnd',
14   drop: 'onDrop',
15   'drag-enter': 'onDragEnter',
```

```
16     'drag-leave': 'onDragLeave',
17     'drop-ready': 'onDropReady'
18 }
19
20 export const SmoothDndContainer = defineComponent({
21     name: 'SmoothDndContainer',
22     setup() {
23         return {
24             container: null as SmoothDnD | null
25         }
26     },
27     mounted() {
28         // emit events
29         const options: any = Object.assign({}, this.$props)
30         for (const key in eventEmitterMap) {
31             const eventKey = key as EventKey
32             options[eventEmitterMap[eventKey]] = (props: any) => {
33                 this.$emit(eventKey, props)
34             }
35         }
36         const containerElement = this.$refs.container || this.$el
37         this.container = smoothDnD(containerElement, options)
38     },
39     unmounted() {
40         if (this.container) {
41             try {
42                 this.container.dispose()
43             } catch {
44                 // ignore
45             }
46         }
47     },
48     emits: ['drop', 'drag-start', 'drag-end', 'drag-enter', 'drag-leave', 'drop-
ready'],
49     props: {
50         orientation: { type: String, default: 'vertical' },
51         removeOnDropOut: { type: Boolean, default: false },
52         autoScrollEnabled: { type: Boolean, default: true },
53         animationDuration: { type: Number, default: 250 },
54         behaviour: String,
55         groupName: String,
56         dragHandleSelector: String,
57         nonDragAreaSelector: String,
58         lockAxis: String,
59         dragClass: String,
60         dropClass: String,
61         dragBeginDelay: Number,
```

```

62     getChildPayload: Function,
63     shouldAnimateDrop: Function,
64     shouldAcceptDrop: Function,
65     getGhostParent: Function,
66     dropPlaceholder: [Object, Boolean],
67     tag: {
68       validator: validateTagProp,
69       default: 'div'
70     }
71   },
72   render() {
73     const tagProps = getTagProps(this)
74     return h(
75       tagProps.value,
76       Object.assign({}, { ref: 'container' }, tagProps.props),
77       this.$slots.default?.(())
78     )
79   }
80 })
81

```

SmoothDndDraggable.ts

```

1  import { defineComponent, h } from 'vue'
2  import { constants } from 'smooth-dnd'
3  import { getTagProps, validateTagProp } from './utils'
4
5  export const SmoothDndDraggable = defineComponent({
6    name: 'SmoothDndDraggable',
7    props: {
8      tag: {
9        validator: validateTagProp,
10       default: 'div'
11     }
12   },
13   render: function () {
14     //wrap child
15     const tagProps = getTagProps(this, constants.wrapperClass)
16     return h(tagProps.value, Object.assign({}, tagProps.props),
17       this.$slots?.default?.(())
18     )
19   }
20 })
21

```

课程总结

本次课我们重点为大家介绍了无代码平台相关需求、Vue3 项目从零到一的架构思想，另外在技术选型方面我们都需要如何思考，最后带大家整体实现了无代码平台基础框架包含导航、物料以及相关配置。

希望大家课后，反复多学习思考，重点学习项目设计的思想，下次换个其他项目你也有能力完全主导设计开发，这样你的核心竞争力才能提高。

共勉！

其他资料

 <https://form.making.link/>

可视化低代码表单设计器 - FormMaking

基于 Vue 的可视化低代码表单设计器，赋能企业实现低代码开发模式；帮助开发者从传统枯燥的表单代码中解放出来，更多关注业务，快速提高效率，节省研发成本。在OA系统、考试系统、报表系统、流程管理等系统中应用广泛。

alibaba/lowcode-engine

An enterprise-class low-code technology stack with scale-out design / 一套面向扩展设计的企业级低代码技术体系

89 Contributors 95 Used by 15k Stars 3k Forks



<https://github.com/alibaba/lowcode-engine>

GitHub - alibaba/lowcode-engine: An enterprise-class low-code technology stack with scale-out design

An enterprise-class low-code technology stack with scale-out design / 一套面向...

sahadev/vue-component-creator-ui

拖拽式Vue组件代码生成编辑器 (VCC)

2 Contributors 5 Issues 2 Discussions 789 Stars 216 Forks



<https://github.com/sahadev/vue-component-creator-ui>

GitHub - sahadev/vue-component-creator-ui: 拖拽式Vue组件代码生成编辑器 (VCC)

拖拽式Vue组件代码生成编辑器 (VCC) . Contribute to sahadev/vue-component-...